AME 5101 - Applied Linear Algebra
Practice Exercises: Reference Answers

Try each part on your own first. The blue text lines give hints on how to start thinking. Then compare your working with the answers below.

Q1. Attention

(a) *Hint: Imagine a library. You walk in with a question, every book has a label on its spine, and what you finally read is the content inside the book.*

The three matrices play three different roles.

- **Query** (column t_x of $\mathbf{Q}$): what the token at position t_x is *looking for*.
- **Key** (column t_z of $\mathbf{K}$): what the token at position t_z *advertises*, so that others can match with it.
- **Value** (column t_z of $\mathbf{V}$): the *content* that token t_z hands over if it is chosen.

A query and a key are compared by a dot product, $S[t_z, t_x] = \mathbf{k}_{t_z}^\top \mathbf{q}_{t_x}$. A large score means "this key matches this query well". The softmax of each column turns the scores into weights that are positive and add up to 1. The output for position t_x is the weighted average of the values, $\sum_{t_z} w_{t_z,t_x} \mathbf{v}_{t_z}$.

Queries and keys must have the same length d_{attn}, otherwise the dot product is not defined. Values can have a different length d_{out}. Using three separate learned matrices lets the model use one view of a token for asking, another for being found, and a third for giving.

(b) *Hint: Ask what GPT is trained to do. Which token is the "correct answer" for position t_x?*

A decoder-only model like GPT is trained to predict the *next* token at every position, and all positions are processed together in one pass. The correct answer for position t_x is the token at position $t_x + 1$.

If position t_x could look at later tokens, it could simply copy its own answer. The loss would become almost zero and the model would learn nothing useful. Also, when we generate text, the future tokens do not exist yet.

The causal mask $\text{Mask}[t_z, t_x] = [\![t_z \le t_x]\!]$ prevents this. Wherever it is false (a key in the future of the query), line 5 sets the score to $-\infty$. Since $e^{-\infty} = 0$, that key gets weight exactly zero, and the softmax shares the whole weight 1 among the visible tokens only. So training on all positions at once behaves like generating one token at a time, from left to right.

(c) *Hint: Look at one score. It is a sum of d_{attn} products. How big does such a sum become?*

A score is $S = \sum_{i=1}^{d_{\text{attn}}} q_i k_i$. Suppose the entries of a query and a key are independent, with mean 0 and variance 1. Then each product $q_i k_i$ has mean 0 and variance 1, and the sum of d_{attn} independent terms has variance d_{attn}. So a typical score has size about $\sqrt{d_{\text{attn}}}$.

Dividing by $\sqrt{d_{\text{attn}}}$ divides the standard deviation by $\sqrt{d_{\text{attn}}}$, so the variance becomes $d_{\text{attn}}/d_{\text{attn}} = 1$, whatever the width. For example, with $d_{\text{attn}} = 64$ the raw scores are typically around ± 8, and after scaling they are around ± 1.

(d) *Hint: Try a small example. Put the scores $(8,0)$ into the softmax, and then the scaled scores $(1,0)$.*

Without the scaling the scores are large, and the softmax becomes almost one-hot.

$$\text{softmax}(8,0) \approx (0.9997,\ 0.0003), \qquad \text{softmax}(1,0) \approx (0.731,\ 0.269).$$

Two things go wrong when the softmax is saturated like this.

- Almost all the weight goes to one token, so the attention is very "hard" and the average over values is almost a single value.
- The gradient through the softmax is proportional to $q(1-q)$. For the saturated case this is $0.9997 \times 0.0003 \approx 0.0003$, but for the scaled case it is $0.731 \times 0.269 \approx 0.197$. A gradient that is nearly zero means learning is very slow or stuck.

The scaling by $\sqrt{d_{\text{attn}}}$ keeps the softmax in a healthy range, where the gradients are not tiny.

(e) *Hint: First write $\mathbf{K}^\top$ (it is 3×2) and multiply. Remember that row = key position and column = query position.*

$$\mathbf{K}^\top = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

$$\mathbf{S} = \mathbf{K}^\top \mathbf{Q} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 0 \\ 1 & 0 & 1 \\ 2 & 2 & 1 \end{pmatrix}.$$

For example, $S[1,2] = \mathbf{k}_1 \cdot \mathbf{q}_2 = (1,0) \cdot (2,0) = 2$, and $S[3,1] = \mathbf{k}_3 \cdot \mathbf{q}_1 = (1,1) \cdot (1,1) = 2$.

The mask keeps entry (t_z, t_x) only when $t_z \le t_x$, that is, on and above the diagonal. Every entry below the diagonal becomes $-\infty$:

$$\text{masked } \mathbf{S} = \begin{pmatrix} 1 & 2 & 0 \\ -\infty & 0 & 1 \\ -\infty & -\infty & 1 \end{pmatrix}.$$

Effect, column by column: query 1 sees only key 1; query 2 sees keys 1 and 2; query 3 sees all three keys. No token can attend to a later token.

(f) *Hint: Look only at column 2 of the matrix, since that column belongs to the query at $t_x = 2$. Go in order: raw scores, then the mask, then the scaling, then the softmax.*

Step 1: the raw scores. Column 2 of $\mathbf{S}$ holds the scores of query $\mathbf{q}_2 = (2,0)$ against the three keys:

$$\begin{aligned} \mathbf{k}_1 \cdot \mathbf{q}_2 &= (1,0) \cdot (2,0) = 2, \\ \mathbf{k}_2 \cdot \mathbf{q}_2 &= (0,1) \cdot (2,0) = 0, \\ \mathbf{k}_3 \cdot \mathbf{q}_2 &= (1,1) \cdot (2,0) = 2. \end{aligned}$$

So the raw column is $(2,\ 0,\ 2)^\top$. Notice that without the mask, keys 1 and 3 would tie.

Step 2: the mask. Key 3 is in the future of position 2, so its score is replaced by $-\infty$. The column becomes $(2,\ 0,\ -\infty)^\top$, and key 3 will get weight exactly 0.

Step 3: the scaling. Dividing by $\sqrt{d_{\text{attn}}} = \sqrt{2}$ gives $(1.414,\ 0,\ -\infty)^\top$.

Step 4: the softmax. The exponentials are $e^{1.414} \approx 4.11$, $e^0 = 1$ and $e^{-\infty} = 0$. Their sum is about 5.11, so

$$\text{weights} \approx \left(\tfrac{4.11}{5.11},\ \tfrac{1}{5.11},\ 0\right) = (0.80,\ 0.20,\ 0).$$

The weights add up to 1, as a softmax must. (Without the scaling, the weights would be $\approx (0.88,\ 0.12,\ 0)$. The order does not change.)

Conclusion. **Key** 1 receives the larger attention weight. It gets about four times the weight of key 2, because the ratio of the two weights is $e^{1.414-0} \approx 4.1$. The softmax is increasing, so a larger score always gives a larger weight.

Why key 1 scores higher. A dot product measures how well two vectors point in the same direction. The query $\mathbf{q}_2 = (2, 0)$ points along the first axis, and so does $\mathbf{k}_1 = (1, 0)$, so they are perfectly aligned and the score is large. The key $\mathbf{k}_2 = (0, 1)$ is perpendicular to $\mathbf{q}_2$, so its score is 0.

What this means for the output. The output for position 2 is $0.80\,\mathbf{v}_1 + 0.20\,\mathbf{v}_2 + 0 \cdot \mathbf{v}_3$. It is made mostly of the value of token 1, and token 3 contributes nothing because it lies in the future.

Q2. Multi-head attention

(a) *Hint: A single softmax gives each position only one set of weights adding up to 1. Can one set of weights serve many different purposes at once?*

Each head has its own $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$, so each head computes its own attention pattern: for every query, its own weights over the keys. A single head gives each position just one weighted average, and it has to blend every relation it needs into that one average.

With several heads, different heads can attend to different tokens for different reasons at the same time. For example, one head may look at the previous token, another at a related word in the same phrase, and another at an earlier mention of a name. Each head works in its own lower-dimensional subspace, and $\mathbf{W}_o$ then combines these different views.

(b) *Hint: Stack first, then multiply. Keep track of the shapes at every step.*

Line 2 (stack vertically). Each $\mathbf{Y}^h$ is $d_{\text{mid}} \times \ell_x = 2 \times 2$. Stacking the two gives

$$\mathbf{Y} = \begin{pmatrix} \mathbf{Y}^1 \\ \mathbf{Y}^2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

Shapes: $\mathbf{Y} \in \mathbb{R}^{(Hd_{\text{mid}}) \times \ell_x} = \mathbb{R}^{4\times 2}$ and $\mathbf{W}_o \in \mathbb{R}^{d_{\text{out}} \times Hd_{\text{mid}}} = \mathbb{R}^{2\times 4}$. The inner sizes match, so $\mathbf{W}_o\mathbf{Y}$ is 2×2.

Line 3. Row 1 of $\mathbf{W}_o$ is $(1, 0, 1, 0)$. It adds row 1 and row 3 of $\mathbf{Y}$:

$$(1, 0) + (0, 1) = (1, 1).$$

Row 2 of $\mathbf{W}_o$ is $(0, 1, 0, -1)$. It takes row 2 minus row 4 of $\mathbf{Y}$:

$$(2, 1) - (1, 1) = (1, 0).$$

So $\mathbf{W}_o\mathbf{Y} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. The bias term $\mathbf{b}_o\mathbf{1}^\top$ adds 0 to every entry of row 1 and 1 to every entry of row 2:

$$\tilde{\mathbf{V}} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 2 & 1 \end{pmatrix}.$$

Which heads contribute? Rows 1 and 2 of $\mathbf{Y}$ belong to head 1, and rows 3 and 4 belong to head 2. Output row 1 uses rows 1 and 3 (one row from each head). Output row 2 uses rows 2 and 4 (again one from each head). So **both heads contribute to every row** of $\tilde{\mathbf{V}}$.

(c) *Hint: Count multiply–adds in three places: the projections, the scores, and the weighted sum of values. Use $h\, d_k = d_{\text{model}}$.*

Compare h heads of width $d_k = d_{\text{model}}/h$ with one head of width d_{model}, for a sequence of length ℓ.

- **Projections.** One head needs a $d_k \times d_{\text{model}}$ matrix for each of $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$. For h heads this is $h\, d_k \times d_{\text{model}} = d_{\text{model}} \times d_{\text{model}}$ in total, exactly the size of the matrix of a single wide head. So the number of parameters and multiply–adds, $d_{\text{model}}^2\, \ell$, is the same.
- **Scores.** Each head computes ℓ^2 dot products of length d_k. For h heads: $h\, \ell^2 d_k = \ell^2 d_{\text{model}}$, the same as one wide head.
- **Weighted sum of values.** Each head again costs $\ell^2 d_k$, so the total is $\ell^2 d_{\text{model}}$, the same as one wide head.

The extra work is small: the softmax is done h times, and we store h score matrices of size $\ell \times \ell$ instead of one. So the arithmetic stays close to that of a single wide head, and only the memory for the scores grows by a factor h.

(d) *Hint: Look at the shapes in part (b). What does $\mathbf{Y}$ look like before $\mathbf{W}_o$, and after?*

$\mathbf{Y}$ is made of H blocks of rows, and each block was produced by one head on its own. $\mathbf{W}_o$ does two jobs.

- It **mixes the heads**: every output row is a linear combination of rows from all the heads (as we saw in part (b)), so information from different heads is combined.
- It **fixes the output size**: it maps the Hd_{mid} rows to d_{out} rows. In a transformer $d_{\text{out}} = d_{\text{model}}$, which the residual connection requires.

If $\mathbf{W}_o$ were omitted, the heads' outputs would stay in separate blocks, with no interaction between heads in this layer, and the output would have Hd_{mid} rows whether or not that suits the next layer.

(e) *Hint: This is the same idea as in part (d), seen from the other side: ask what you would be handing to the next layer.*

If MHAttention returned the stacked $\mathbf{Y}$ directly, we would lose three things.

- **Mixing of heads.** Each head would stay in its own block of rows. Nothing in this layer lets the information of one head combine with another's.
- **Control of the size.** The output would have Hd_{mid} rows (4 in our example, instead of the 2 we get from $\mathbf{W}_o$). We could no longer choose d_{out} freely, for example to match the width needed for the residual connection.
- **Learnable parameters.** The matrix $\mathbf{W}_o$ and the bias $\mathbf{b}_o$ are extra parameters that the layer can learn. We would lose that flexibility.

Q3. Cross-entropy loss on three examples

(a) *Hint: The target is one-hot, so in the product $\prod_k q_{ik}^{y_{ik}}$ every factor with $y_{ik} = 0$ is $q^0 = 1$.*

Only the factor with $y_{ik} = 1$ survives, so $P(c_i \mid \mathbf{x}_i, \theta)$ is simply the probability given to the true class.

$$P_1 = 0.5^1 \cdot 0.3^0 \cdot 0.2^0 = 0.5,$$
$$P_2 = 0.1^0 \cdot 0.6^1 \cdot 0.3^0 = 0.6,$$
$$P_3 = 0.25^0 \cdot 0.25^0 \cdot 0.5^1 = 0.5.$$

(b) *Hint: The examples are independent, so probabilities multiply.*

$$\text{likelihood} = P_1 P_2 P_3 = 0.5 \times 0.6 \times 0.5 = 0.15.$$

(c) *Hint: The mean loss is the average of $-\ln P_i$. You are given $\ln 2, \ln 3, \ln 5$, so write $0.6 = 3/5$.*

$$\bar{L} = -\frac{1}{3}(\ln 0.5 + \ln 0.6 + \ln 0.5).$$

Now $\ln 0.5 = -\ln 2 = -0.693$ and $\ln 0.6 = \ln 3 - \ln 5 = 1.099 - 1.609 = -0.510$. Therefore

$$\bar{L} = \frac{0.693 + 0.510 + 0.693}{3} = \frac{1.896}{3} = 0.632.$$

Check: $\bar{L} = -\frac{1}{3}\ln(\text{likelihood}) = -\frac{1}{3}\ln 0.15 = \frac{1.897}{3} \approx 0.632$. ✓

Q4. Surprisal and entropy

(a) *Hint: (i) Count how many times each outcome appears in N draws. (ii) Just substitute the probabilities into $-\sum q_k \ln q_k$.*

(*i*) In N independent draws from $\mathbf{q}$, outcome k appears about $n_k \approx Nq_k$ times (for large N). Each time it appears, it carries surprisal $-\ln q_k$. So

$$\text{average surprisal per draw} = \frac{1}{N}\sum_k n_k(-\ln q_k) \approx \sum_k q_k(-\ln q_k) = H(\mathbf{q}).$$

(*ii*) If all K outcomes are equally likely, $q_k = 1/K$, so

$$H(\mathbf{q}) = -\sum_{k=1}^{K} \frac{1}{K} \ln \frac{1}{K} = -K \cdot \frac{1}{K} \cdot (-\ln K) = \ln K.$$

If one outcome has probability 1, its term is $-1 \cdot \ln 1 = 0$. All other outcomes have $q_k = 0$, and their terms are $0 \cdot \ln 0$, which we define as 0 (the limit of $x \ln x$ as $x \to 0$). So $H(\mathbf{q}) = 0$.

(b) *Hint: First get the three surprisals, then average them using the probabilities as weights.*

With $\mathbf{q} = (0.5, 0.25, 0.25)$:

$$\begin{aligned} \text{outcome 1:} \quad & -\ln 0.5 = \ln 2 = 0.693, \\ \text{outcomes 2 and 3:} \quad & -\ln 0.25 = \ln 4 = 2 \ln 2 = 1.386 \text{ nats each.} \end{aligned}$$

The entropy is the average of these surprisals, weighted by $\mathbf{q}$:

$$\begin{aligned} H(\mathbf{q}) &= 0.5(0.693) + 0.25(1.386) + 0.25(1.386) \\ &= 0.3465 + 0.3465 + 0.3465 \approx 1.04 \text{ nats.} \end{aligned}$$

Check: the largest possible entropy for $K = 3$ is $\ln 3 = 1.099$, and our 1.04 is a little below it, as expected for a distribution that is not uniform. ✓

Q5. Surprisal and negative log-likelihood

(a) *Hint: (i) Independent means probabilities multiply. (ii) Write the likelihood of the data set, then take $-\ln$.*

(*i*) For independent outcomes A and B, $P(A \text{ and } B) = P(A)P(B)$. So

$$h(A, B) = -\ln(P(A)P(B)) = -\ln P(A) - \ln P(B) = h(A) + h(B).$$

The surprisals add.

(*ii*) The examples are independent, so the likelihood is $\prod_{i=1}^{N} P(c_i \mid \mathbf{x}_i, \theta)$. The negative log-likelihood is

$$\text{NLL} = -\ln \prod_{i=1}^{N} P_i = \sum_{i=1}^{N} (-\ln P_i) = \sum_{i=1}^{N} h(c_i).$$

This is the sum of the surprisals of the observed labels, that is, the total surprisal.

(b) *Hint: Take the log of each probability, add them up (that is the log-likelihood), then divide the negative by 3.*

For $q_{c_1} = 0.5$, $q_{c_2} = 0.8$, $q_{c_3} = 0.25$:

$$\ell = \ln 0.5 + \ln 0.8 + \ln 0.25.$$

Here $\ln 0.5 = -0.693$, $\ln 0.25 = -2 \ln 2 = -1.386$, and $\ln 0.8 = \ln 4 - \ln 5 = 1.386 - 1.609 = -0.223$. So

$$\ell = -0.693 - 0.223 - 1.386 = -2.302 \text{ nats.}$$

The average cross-entropy loss is

$$\bar{L} = -\frac{\ell}{3} = \frac{2.302}{3} \approx 0.767 \text{ nats.}$$

Check: the likelihood is $0.5 \times 0.8 \times 0.25 = 0.1$, and $\ln 0.1 = -2.303$. ✓

Q6. The loss in terms of the logits

(a) *Hint: Write q_c using the softmax definition, take $-\ln$, and use $\ln \frac{a}{b} = \ln a - \ln b$.*

The softmax gives $q_c = \dfrac{e^{z_c}}{\sum_j e^{z_j}}$. Therefore

$$L = -\ln q_c = -\Big(\ln e^{z_c} - \ln \sum_j e^{z_j}\Big) = -z_c + \ln \sum_j e^{z_j}.$$

(b) *Hint: Factor e^m out of every term of the sum.*

Since $e^{z_j} = e^m e^{z_j - m}$,

$$\sum_j e^{z_j} = e^m \sum_j e^{z_j - m} \quad \Longrightarrow \quad \ln \sum_j e^{z_j} = m + \ln \sum_j e^{z_j - m}.$$

Why this avoids overflow: $z_j - m \le 0$, so every $e^{z_j - m}$ lies in $(0, 1]$, and the largest term is exactly $e^0 = 1$. The sum is therefore between 1 and K, a safe number. Without the shift, a logit such as $z = 1000$ gives e^{1000}, which is too large for a computer (a 64-bit float overflows beyond about $e^{709.8}$). So we compute $L = -z_c + m + \ln \sum_j e^{z_j - m}$.

Q7. Two classes: from logits to loss

(a) *Hint: Look at the numbers. Can $\ln 3 = 1.099$ be a probability? Do the two numbers of an example add up to 1?*

Logits are raw scores. They can be negative or bigger than 1, and they do not add up to 1. For example $\ln 3 = 1.099 > 1$, and the logits $(\ln 3, 0)$ add up to 1.099. To turn them into probabilities we apply the softmax,

$$q_k = \frac{e^{z_k}}{\sum_j e^{z_j}},$$

which makes every q_k positive and makes them add up to 1.

(b) *Hint: Since $e^{\ln 3} = 3$, $e^{\ln 2} = 2$ and $e^0 = 1$, the exponentials are simple numbers.*

$$\begin{aligned}
&\text{Example 1: } e^{\mathbf{z}} = (3, 1),\ \text{sum } 4,\ \mathbf{q} = (0.75,\ 0.25),\\
&\text{Example 2: } e^{\mathbf{z}} = (1, 3),\ \text{sum } 4,\ \mathbf{q} = (0.25,\ 0.75),\\
&\text{Example 3: } e^{\mathbf{z}} = (2, 2),\ \text{sum } 4,\ \mathbf{q} = (0.5,\ 0.5).
\end{aligned}$$

(c) *Hint: The loss uses only the probability of the* true *class, so pick q_c for each example.*

$$\begin{aligned}
&\text{Example 1 } (c = 1) : L_1 = -\ln 0.75 = \ln 4 - \ln 3 = 1.386 - 1.099 = 0.287,\\
&\text{Example 2 } (c = 1) : L_2 = -\ln 0.25 = \ln 4 = 1.386,\\
&\text{Example 3 } (c = 2) : L_3 = -\ln 0.5 = \ln 2 = 0.693.
\end{aligned}$$

The mean loss is

$$\bar{L} = \frac{0.287 + 1.386 + 0.693}{3} = \frac{2.366}{3} \approx 0.789.$$

Does it make sense? Example 1 is correct and fairly confident, so its loss is small. Example 2 puts only 0.25 on the true class, so its loss is the largest. Example 3 is a 50–50 guess, which costs $\ln 2$.

Q8. Mean loss for a data set

(a) *Hint: Look at the exponent y_{ik}. When is it 1, and when is it 0?*

The target is one-hot: $y_{ik} = 1$ only for $k = c_i$, and $y_{ik} = 0$ for all other k. Every factor with $y_{ik} = 0$ equals $q_{ik}^0 = 1$, so it does not change the product. Only the factor $k = c_i$ remains:

$$\prod_{k=1}^{K} q_{ik}^{y_{ik}} = q_{ic_i}.$$

This is exactly the probability the network gives to the true class, $P(c_i \mid \mathbf{x}_i, \theta)$.

(b) *Hint: Follow the three steps: likelihood, then logarithm, then negation. Finally divide by n.*

- *Likelihood.* The examples are independent, so $\mathcal{L}(\theta) = \prod_{i=1}^{n} P(c_i \mid \mathbf{x}_i, \theta) = \prod_{i=1}^{n} q_{ic_i}$, using part (a).
- *Logarithm.* $\ln \mathcal{L}(\theta) = \sum_{i=1}^{n} \ln q_{ic_i}$.
- *Negation.* $\text{NLL} = -\sum_{i=1}^{n} \ln q_{ic_i} = \sum_{i=1}^{n} (-\ln q_{ic_i})$.

Dividing by n gives the mean loss

$$\bar{L} = \frac{1}{n} \sum_{i=1}^{n} (-\ln q_{ic_i}).$$

Minimising $\bar{L}$ is the same as maximising the likelihood.

Q9. A target that is not one-hot

(a) *Hint: First the softmax: exponentiate the logits (they are $\ln 2, \ln 2, 0$), then divide by the sum. For the loss, note that $p_3 = 0$.*

Here $e^{\mathbf{z}} = (2, 2, 1)$ with sum 5, so

$$\mathbf{q} = (0.4,\ 0.4,\ 0.2).$$

The loss is the cross-entropy

$$L = H(\mathbf{p}, \mathbf{q}) = -\sum_{k} p_k \ln q_k = -(0.5 \ln 0.4 + 0.5 \ln 0.4 + 0 \cdot \ln 0.2) = -\ln 0.4.$$

Now $-\ln 0.4 = \ln 2.5 = \ln 5 - \ln 2 = 1.609 - 0.693 = 0.916$. So $L \approx 0.916$. The third class has $p_3 = 0$, so q_3 does not appear in the loss directly.

(b) *Hint: For the softmax with cross-entropy, the gradient with respect to the logits is* $\mathbf{q} - \mathbf{p}$*. Subtract entry by entry.*

$$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{q} - \mathbf{p} = (0.4 - 0.5,\ 0.4 - 0.5,\ 0.2 - 0) = (-0.1,\ -0.1,\ 0.2).$$

The entries add up to 0, as they should. A gradient step moves against the gradient, so it raises z_1 and z_2 (the classes the annotator likes) and lowers z_3.

Q10. Loss at the start of training

(a) *Hint: If all the logits are the same number* z*, what happens to the numerator and denominator of the softmax?*

Let every logit be equal to z. Then

$$q_k = \frac{e^z}{\sum_{j=1}^{K} e^z} = \frac{e^z}{K\, e^z} = \frac{1}{K} \quad \text{for every } k.$$

(b) *Hint: Use part (a). It does not matter which class is the true one, because all* q_k *are equal.*

For any true class c, $q_c = 1/K$, so

$$L = -\ln q_c = -\ln \frac{1}{K} = \ln K.$$

With small random weights the logits are close to zero, so at the start $\mathbf{q} \approx (1/K, \dots, 1/K)$ and the loss is about $\ln K$. For $K = 10$ this is $\ln 10 = 2.303$. This is a useful sanity check when you train a network: if the first loss is far from $\ln K$, something is wrong.

Q11. Gradient of the loss with respect to a logit

(a) *Hint: Differentiate the two terms of* $L = -z_c + \ln \sum_j e^{z_j}$ *separately. In the second term use the chain rule for* $\ln u$*.*

First term. $\dfrac{\partial}{\partial z_k}(-z_c) = -1$ if $k = c$, and 0 otherwise. Since $y_k = 1$ for $k = c$ and 0 otherwise, this is $-y_k$.

Second term. Let $u = \sum_j e^{z_j}$. By the chain rule,

$$\frac{\partial}{\partial z_k} \ln u = \frac{1}{u} \frac{\partial u}{\partial z_k} = \frac{e^{z_k}}{\sum_j e^{z_j}},$$

because only the term $j = k$ of the sum depends on z_k. This is exactly q_k.

(b) *Hint: Add the two pieces from part (a).*

$$\frac{\partial L}{\partial z_k} = q_k - y_k, \qquad \text{that is,} \qquad \frac{\partial L}{\partial \mathbf{z}} = \mathbf{q} - \mathbf{y}.$$

This is neat. For the true class, $q_c - 1 \le 0$, so gradient descent pushes z_c up. For every other class, $q_k > 0$, so it pushes z_k down.

Q12. Gradients of the weights and biases of a layer

(a) *Hint: The loss depends on W_{ij} only through z_i. Write the chain rule and see which terms vanish.*

By the chain rule,

$$\frac{\partial L}{\partial W_{ij}} = \sum_m \frac{\partial L}{\partial z_m} \frac{\partial z_m}{\partial W_{ij}}.$$

Since $z_m = \sum_j W_{mj} a_j + b_m$, the weight W_{ij} appears only in z_i, with $\partial z_i / \partial W_{ij} = a_j$, and $\partial z_m / \partial W_{ij} = 0$ for $m \neq i$. Only the term $m = i$ survives:

$$\frac{\partial L}{\partial W_{ij}^{(l)}} = \delta_i^{(l)} \, a_j^{(l-1)}.$$

In the same way $\partial z_i / \partial b_i = 1$ and the other terms vanish, so

$$\frac{\partial L}{\partial b_i^{(l)}} = \delta_i^{(l)}.$$

(b) *Hint: Look at the entries $\delta_i a_j$ as a table with rows i and columns j. That is an outer product.*

The (i, j) entry of $\nabla_{\mathbf{W}^{(l)}} L$ is $\delta_i a_j$, so

$$\nabla_{\mathbf{W}^{(l)}} L = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^\top, \qquad \nabla_{\mathbf{b}^{(l)}} L = \boldsymbol{\delta}^{(l)}.$$

The shape is $(n_l \times 1)(1 \times n_{l-1}) = n_l \times n_{l-1}$, the same as $\mathbf{W}^{(l)}$.

A small example: if $\boldsymbol{\delta} = (2, -1)^\top$ and $\mathbf{a} = (3, 0, 1)^\top$, then

$$\boldsymbol{\delta} \mathbf{a}^\top = \begin{pmatrix} 6 & 0 & 2 \\ -3 & 0 & -1 \end{pmatrix}.$$

Notice that column j is $a_j \boldsymbol{\delta}$, so the column for $a_2 = 0$ is all zero. This is why an inactive ReLU unit in the previous layer gives zero gradients for the weights coming out of it.